УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
НОВИ САД

Веб апликација за подршку рада гасних котлова за грејање објеката

Предмет: *Софтверска реализација експлоатације електроенергетских система*

Професор:
Проф. др Александар Бошковић

Студент:
Милорад Максић

Нови Сад, Март, 2026.

Садржај

1. Увод 1
1.1. Циљ пројекта 1
1.2. Опис задатка 1
1.3. Обим и ограничења система 1
1.4. Преглед корисничких улога 2
2. Опис система 2
2.1. Главна идеја рјешења 2
2.2. Опис процеса у систему (логика рада) 2
2.3. Дијаграм случајева употребе (Use Case Diagram) 4
3. Архитектура апликације 5
3.1. Преглед архитектуре (frontend → backend → database → API интеграције) 5
3.2. Опис комуникације (REST API, токови података) 5
4. Коришћене технологије 6
4.1. Frontend 6
4.2. Backend 7
4.3. База података 7
4.4. Интеграције са API-јима 8
5. Структура базе података 9
5.1. ER дијаграм базе 9
5.2. Опис сваке табеле 9
5.3. Релације између табела 9
5.4. Кључни атрибути и индекси 10
6. Коришћени упити 10
6.1. CRUD упити за главне ентитете 10
6.2. Кључни SELECT упити 11
6.3. Примјери REST позива 12
7. Функционалности система 13
7.1. Преглед свих функционалности 13
7.2. Опис логике рада 13
Главни циклус рада система 13
Термодинамички модел 14
7.3. Посебне функционалности по улози корисника 14

8. Документација кода 15
8.1. Опис кључних модула 15
8.2. Опис најважнијих класа и функција 15
8.3. Објашњење сложених алгоритама 16
9. Потенцијална унапређења 17
9.1. Техничка унапређења 17
9.2. Нове функционалности 17
9.3. Оптимизације 18
9.4. Могућности за скалабилност 18
10. Закључак 19

1. Увод

1.1. Циљ пројекта

Веб апликација за подршку рада гасних котлова има за циљ да симулира реалистичан систем гријања и омогући корисницима анализу и управљање системима гријања различитих типова објеката, као и процјену потрошње енергије и трошкова гријања. Систем омогућава дефинисање креирање објекта, термичких карактеристика грађевинских елемената и параметара гасног котла, на основу чега се израчунавају топлотни губици и потребна снага за гријање.

1.2. Опис задатка

Задатак овог пројекта јесте да се симулира реалан систем за гријање зграда помоћу гасних котлова. Потребно је имплементирати механизме за креирање објеката, гасних котлова, прикази у листи, прикази на мапи. Један од главних захтјева пројекта јесте имплементација симулације гријања и хлађења зграда као и јачине гасних котлова помоћу П-регулатора.

1.3. Обим и ограничења система

Пројекат подржава: управљање корисницима, управљање зградама, управљање гасним котловима, аутоматску контролу температуре и јачине котлова. Једноставан термодинамички модел, рачунање топлотних губитака. Интергација екстерних API-ја. Имамо историјске податке и анализу, аларми као и неке додатне функционалности попут: експортовања у CSV, неке real-time updates (SignalR WebSocket), графови.

Нека од ограничења: само обичан једноставан P контролер (PID контролер није имплементиран), не постоји физичка интеграција, ово је ипак само симулација (нема SCADA систем). Једноставна термодинамика, модел није напредан, само најосновније. Не постоје неке напредније функционалности, техничке лимитације. Ограничени број API позива ка OpenWeatherMap API-ju

1.4. Преглед корисничких улога

Функционалност / Радња	Администратор	Корисник
Регистрација и пријава	✓	✓
Креирање објеката	✕	✓
Креирање гасних котлова	✕	✓
Управљање сопственим котловима	✕	✓
Преглед свих корисника	✓	✕
Блокирање корисника	✓	✕
Подешавање глобалних параметара	✓	✕
Преглед дијаграма и потрошње	✓	✓
Преглед трошкова грејања	✓	✓
Извоз података (CSV)	✓	✓
Преглед основних информација	✓	✓

Табела 1 Преглед овлашћења по корисничким улогама

2. Опис система

2.1. Главна идеја рјешења

Главна идеја овог рјешења јесте да се симулира рад реалног система за гријање помоћу гасних котлова. Контрола дијелова система, аутоматски позиви за добијање температура, за контролу котлова, њихове јачине. Симулација унтурашње температуре.

2.2. Опис процеса у систему (логика рада)

П-Регулатор:

- Систем учитава све објекте из базе података
- За сваки објекат преузима последњи снимљени податак (BuildingReading)
- Израчунава тренутну унутрашњу температуру користећи термодинамички модел:
    - Toplota generisana: Q_gen = Σ(snaga kotla × efikasnost × 1000)
    - Toplota izgubljena : Q_lost = toplotni gubici objekta
    - Neto tok toplote : Q_net = Q_gen - Q_lost
    - Promjena temperature : ΔT = (Q_net × 60s) / (Termicka masa ×Volumen) × 0.15
- Израчунава грешку: error = zeljena temperatura – trenutna temperatura
- Израчунава корекцију снаге: korekcija = error × Kp (где је Kp = 1.5)
- Прилагођава снагу котлова: nova snaga = trenutna snaga + korekcija
- Ограничава снагу: снага мора бити између 0 и максималне снаге
- Чува нову температуру у бази података

- Шаље ажурирање преко SignalR-а на фронтенд

OpenWeatherMap API:

Систем преузима спољну температуру од OpenWeatherMap API-ја

За сваки објекат:

- Преузима последњи снимљени податак (за очување унутрашње температуре)
- Израчунава топлотне губитке на основу спољне температуре
- Израчунава потребну снагу за грејање
- Процењује дневне трошкове грејања

- Креира нови BuildingReading запис у бази
- Чува: унутрашњу температуру, спољну температуру, топлотне губитке, снагу котлова, трошкове

Корисничке интеракције:

Креирање објекта:

1. Корисник уноси геометријске податке (грејна површина, висина плафона)
2. Корисник (админ) подешава U-вредности за све елементе
3. Корисник поставља жељену температуру
4. Систем чува објекат у базу података
5. Објекат се приказује на мапи

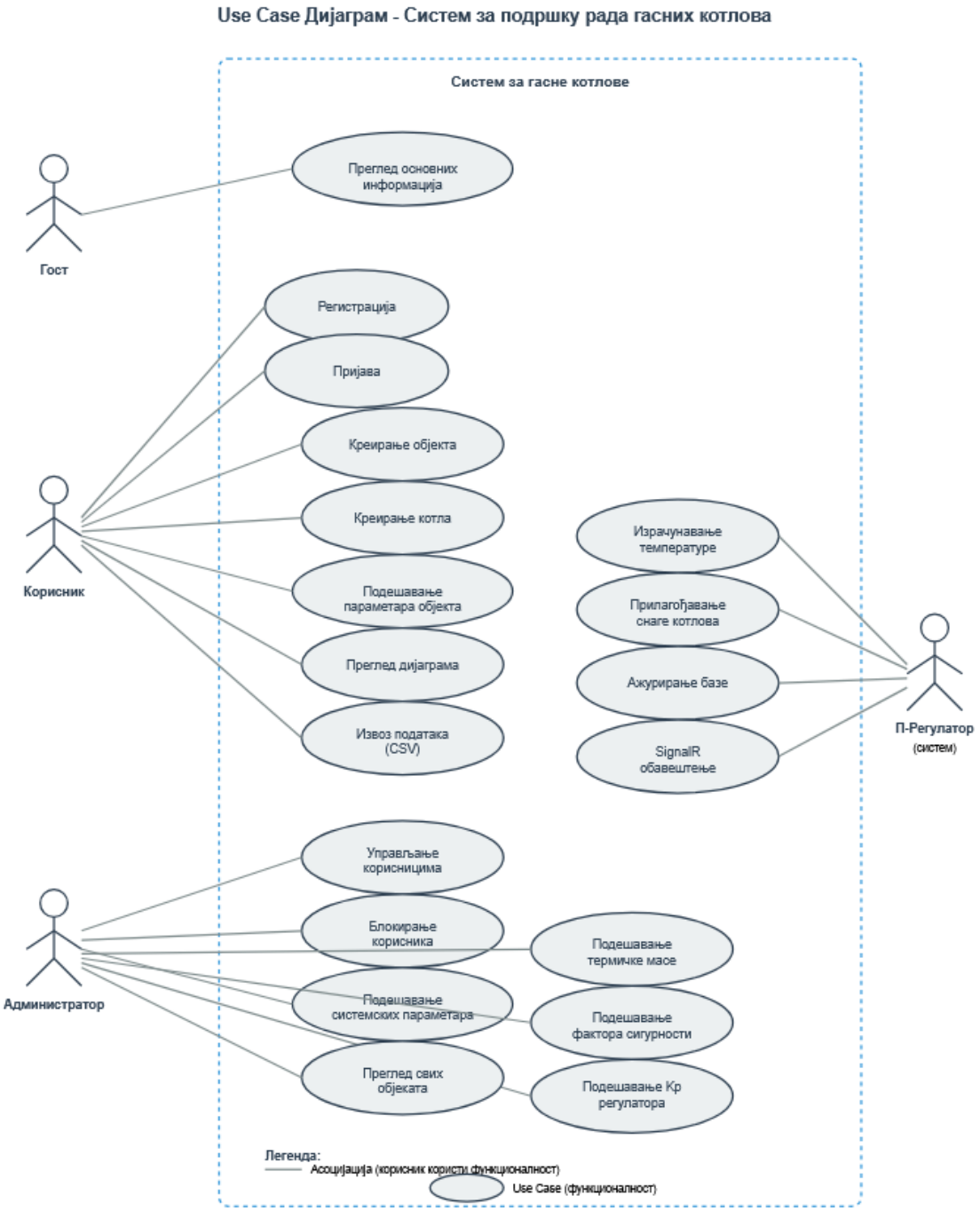
Креирање котла:

1. Корисник бира објекат
2. Корисник уноси име котла, максималну снагу, ефикасност
3. Систем чува котао и повезује га са објектом
4. P-регулатор почиње да управља котлом

Преглед података:

1. Корисник бира објекат
2. Систем приказује тренутне податке (температура, снага котлова)
3. Систем учитава историјске податке из базе
4. Систем генерише дијаграме (Chart.js)
5. Корисник може да извезе податке у CSV

2.3. Дијаграм случајева употребе (Use Case Diagram)



3. Архитектура апликације

3.1. Преглед архитектуре (frontend → backend → database → API интеграције)

Слој	Технологија	Компоненте	Одговорност
1	**React (SPA)**	MapView, Dashboard, Charts, SignalR клијент	Приказ података, интеракција корисника, примање real-time ажурирања
2	**.NET 8 Web API**	Controllers, Services, Repositories, Background Service	Пословна логика, П-регулатор, REST API, SignalR Hub
3	**SQL Server**	Entity Framework Core, Migrations	Трајно чување података — зграде, бојлери, температуре, аларми
4	**Weather API**	OpenWeatherMap / vanjski провајдер	Спољна температура и израчун губитака топлоте (HeatLossWatts)

3.2. Опис комуникације (REST API, токови података)

Систем користи два различита комуникацијска канала: синхрони REST API за CRUD операције и асинхрони WebSocket (SignalR) за real-time ажурирања.

```
ТОК: Корисник → React → .NET API → Repository → SQL Server → Repository → API → Re-
act


Примјер — учитавање зграда:
  1. Корисник отвори MapView страницу
  2. React: GET /api/buildings  (са JWT токеном у хедеру)
  3. BuildingController прима захтјев, провјерава аутентикацију
  4. IBuildingRepository.GetAllAsync() → EF Core → SQL Server
  5. SQL Server враћа листу зграда
  6. Controller серијализује у JSON → 200 OK
  7. React приказује маркере на мапи
```

```
HTTP методе:
  GET    /api/buildings          → Дохвати све зграде
  POST   /api/buildings          → Креирај нову зграду
  PUT    /api/buildings/{id}     → Ажурирај зграду
  DELETE /api/buildings/{id}     → Обриши зграду
  GET    /api/boilers/building/{id} → Бојлери зграде
  POST   /api/auth/login         → Пријава (враћа JWT)
```

SignalR WebSocket — асинхрона real-time комуникација

```
ТОК: Background Service → SignalR Hub → Сви повезани React клијенти


Примјер — ажурирање температуре:
  1. Background Service изврши регулацију (сваких 60s)
  2. Израчуна нову indoorTemp за сваку зграду
  3. _hubContext.Clients.All.SendAsync("IndoorTemperatureUpdated", { ... })
  4. SignalR Hub прослиједи поруку свим повезаним клијентима
  5. React listener прими поруку без поновног учитавања странице
  6. Dashboard и MapView аутоматски ажурирају температуру


Структура SignalR поруке (JSON):
  {
    buildingId:          number,
    buildingName:        string,
    temperature:         number,   // тренутна indoor °C
    desiredTemperature:  number,   // жељена °C
    error:               number,   // грешка (desired - actual)
    outdoorTemperature:  number,   // vanjska °C
    hasBoilers:          boolean,
    timestamp:           DateTime  // UTC
  }
```

4. Коришћене технологије

4.1. Frontend

Кључне технологије:

React – за компонентски базиран интерфејс и ефикасно управљање стањем
TypeScript – за типску сигурност
Leaflet – за географски приказ објеката на мапи
Chart.js – за генерисање дијаграма температуре, снаге котлова и потрошње

Axios – за HTTP комуникацију са бекендом
SignalR Client – за real-time ажурирање података

4.2. Backend

Кључне технологије:

ASP.NET Core – програмски оквир за изградњу REST API-ја
C# – програмски језик
Entity Framework Core – ORM за рад са базом података
SignalR – за real-time комуникацију са фронтендом
Background Services – за периодичне задатке (П-регулатор, снимање података)

Кључни сервиси:

BoilerControlBackgroundService – Извршава се сваки 1 минут. Израчунава тренутну унутрашњу температуру користећи термодинамички модел, извршава П-регулатор алгоритам, прилагођава снагу котлова на основу грешке између жељене и тренутне температуре, и ажурира базу података.

DataRecordingBackgroundService – Извршава се сваких 60 минута. Креира нове BuildingReading ентитете, преузима спољну температуру од OpenWeatherMap API-ја и израчунава топлотне губитке објекта.

HistoricalDataService – Управља креирањем BuildingReading ентитета. Критична имплементација чува претходну унутрашњу температуру (preserves state) како би П-регулатор могао да акумулира промене температуре преко сатних граница.

BuildingCalculatorService – Израчунава топлотне губитке, потребну снагу за грејање и дневне трошкове на основу геометријских и термичких параметара објекта, спољне температуре и ефикасности котлова.

4.3. База података

База података служи за чување корисничких налога, података о објектима, гасним котловима, термичким параметрима, историјским вредностима и израчунатим трошковима. Користи се Microsoft SQL Server као релациона база података.

Кључне табеле:

- Users – корисници система са улогама (Admin, User, Guest)
- BuildingObjects – подаци о објектима (геометрија, термички параметри)

- GasBoilers – подаци о гасним котловима (снага, ефикасност, тренутна снага)
- BuildingReadings – time-series подаци о температури, топлотним губицима и трошковима (сваких 60 минута)
- SystemParameters – глобални параметри система (термичка маса, фактор сигурности, параметри П-регулатора)
- Alarms – систем аларма за недовољан капацитет котлова

Entity Framework Core користи се за мапирање објеката из C# кода на табеле у бази података, омогућавајући једноставан приступ подацима и миграције шеме.

4.4. Интеграције са API-јима

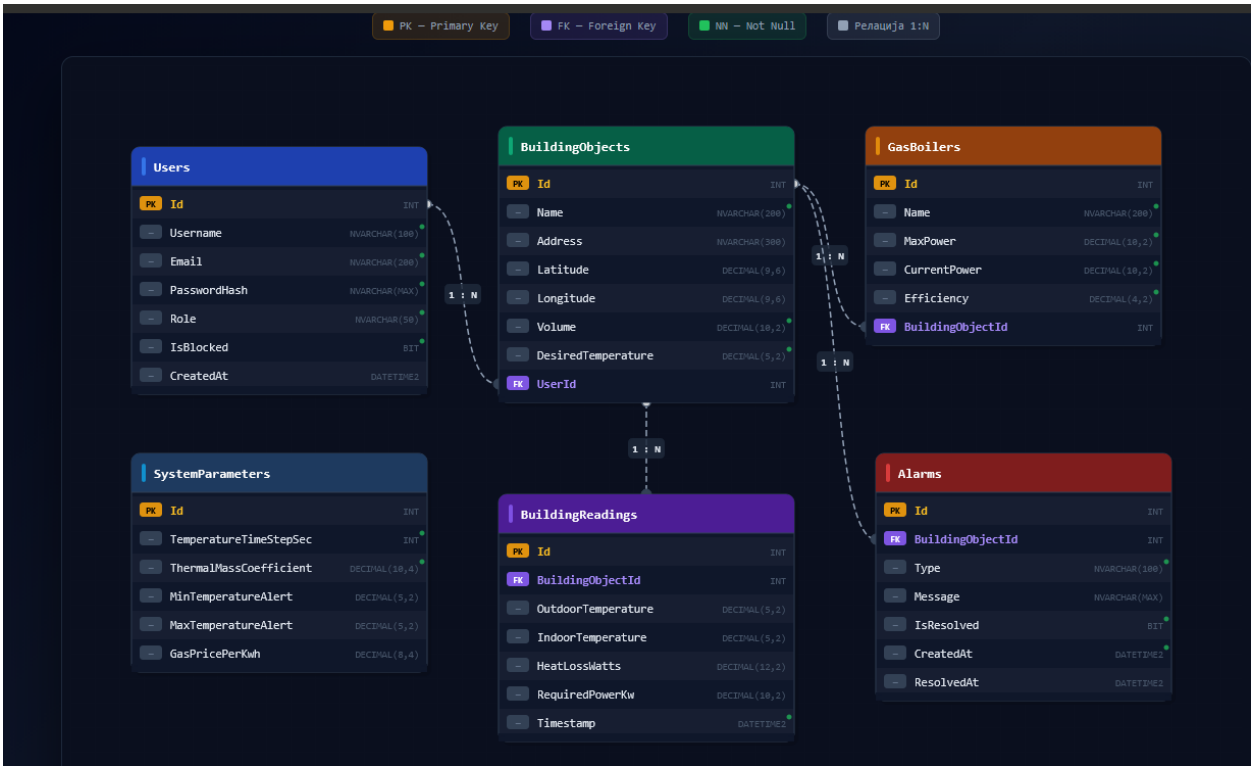
OpenWeatherMap API – за тренутне температуре, облачност и временске прогнозе

Hourly Pricing API (Није имплементирано, не постоји free верзија)– за цијене електричне енергије и гаса у реалном времену

Подаци добијени из ових сервиса интегришу се у модел прорачуна и користе се за приказ резултата у корисничком интерфејсу. Систем аутоматски преузима спољну температуру на почетку сваког сатног циклуса (DataRecordingBackgroundService), док П-регулатор користи ову вредност за израчунавање топлотних губитака и регулацију снаге котлова.

5. Структура базе података

5.1. ER дијаграм базе



5.2. Опис сваке табеле

Users – корисници система са улогама (Admin, User, Guest)

BuildingObjects – подаци о објектима (геометрија, термички параметри)

GasBoilers – подаци о гасним котловима (снага, ефикасност, тренутна снага)

BuildingReadings – time-series подаци о температури, топлотним губицима и трошковима (сваких 60 минута)

SystemParameters – глобални параметри система (термичка маса, фактор сигурности, параметри П-регулатора)

Alarms – систем аларма за недовољан капацитет котлова

5.3. Релације између табела

Users : BuildingObjects → 1:N

BuildingObjects : GasBoilers → 1:N

BuildingObjects : BuildingReadings → 1:N

BuildingObjects : Alarms → 1:N

Један корисник може имати више објеката (1:N)

Један објекат може имати више котлова (1:N)

Један објекат може имати више снимљених података (1:N)

Један објекат може имати више аларма (1:N)

5.4. Кључни атрибути и индекси

Индекси:

- PK_Users
- PK_BuildingObjects, UserId (FK)
- PK_GasBoilers, BuildingId (FK)
- PK_BuildingReadings, BuildingId (FK)
- PK_Alarms, BuildingId (FK)
- PK_SystemParameters

6. Коришћени упити

6.1. CRUD упити за главне ентитете

За упите је коришћен EntityFramework.

Create

```
INSERT INTO BuildingObjects ( Name, UserId, Length, Width, Height, DesiredTemperature, WallUValue, WindowUValue, CeilingUValue, FloorUValue, Latitude, Longitude ) VALUES ( 'Object', 1, 10.0, 8.0, 3.0, 22.0, 0.6, 1.4, 0.25, 0.4, 45.2671, 19.8335 );
```

```
var building = new BuildingObject { Name = "Object", UserId = 1, Length = 10.0, Width = 8.0, Height = 3.0, DesiredTemperature = 22.0, WallUValue = 0.6, WindowUValue = 1.4, CeilingUValue = 0.25, FloorUValue = 0.4, Latitude = 45.2671, Longitude = 19.8335 };
```

```
await _context.BuildingObjects.AddAsync(building);
```

```
await _context.SaveChangesAsync();
```

Read

```
SELECT * FROM BuildingObjects WHERE Id = 1;
```

```
var building = await _context.BuildingObjects .FirstOrDefaultAsync(b => b.Id == 1);
```

Update

```
UPDATE BuildingObjects SET DesiredTemperature = 23.0, WallUValue = 0.5 WHERE Id = 1;
var building = await _context.BuildingObjects .FindAsync(1);
if (building != null)
{
 building.DesiredTemperature = 23.0;
 building.WallUValue = 0.5;
await _context.SaveChangesAsync();
}
```

DELETE

```
DELETE FROM BuildingObjects WHERE Id = 1;
var building = await _context.BuildingObjects .FindAsync(1);
if (building != null)
{
_context.BuildingObjects.Remove(building);
await _context.SaveChangesAsync();
}
```

6.2. Кључни SELECT упити

```
SELECT bo.Id AS BuildingId, bo.Name AS BuildingName, bo.DesiredTemperature, COUNT(gb.Id) AS
BoilerCount, SUM(gb.MaxPower) AS TotalMaxPower, SUM(gb.CurrentPower) AS TotalCurrentPower
FROM BuildingObjects bo LEFT JOIN GasBoilers gb ON bo.Id = gb.BuildingId GROUP BY bo.Id,
bo.Name, bo.DesiredTemperature;
```

```
var result = await _context.BuildingObjects
.Include(b => b.GasBoilers)
.Select(b => new
{
 BuildingId = b.Id,
BuildingName = b.Name,
b.DesiredTemperature,
BoilerCount = b.GasBoilers.Count,
```

```
TotalMaxPower = b.GasBoilers.Sum(gb => gb.MaxPower),
TotalCurrentPower = b.GasBoilers.Sum(gb => gb.CurrentPower)
})
.ToListAsync();
```

6.3. Примјери REST позива

POST /api/auth/login

```
Content-Type: application/json
{
 "username": "korisnik123",
"password": "lozinka123"
}
```

```
Response:
{
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
"userId": 1,
 "username": "korisnik123",
"role": "User"
}
```

GET api/buildings

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..
```

```
Response:
[ { "id": 1, "name": "Obj", "desiredTemperature": 22.0, "latitude": 45.2671, "longitude": 19.8335,
"boilerCount": 2, "totalMaxPower": 48.0 } ]
```

7. Функционалности система

7.1. Преглед свих функционалности

Систем за подршку рада гасних котлова обухвата више функционалних цјелина које корисницима омогућавају дефинисање објеката, прорачун топлотних губитака, управљање гасним котловима, визуелизацију података и анализу трошкова гријања. Апликација је осмишљена тако да подржи различите типове објеката и различите конфигурације система грејања, укључујући могућност коришћења једног или више котлова на истој локацији.

7.2. Опис логике рада

Главни циклус рада система

Систем за подршку рада гасних котлова функционише на основу два паралелна процеса који се извршавају у позадини:

1. П-регулатор (сваки 1 минут): Овај процес представља срце система за аутоматску регулацију. Извршава се сваког минута и врши следеће кораке:

- Преузима све објекте из базе података
- За сваки објекат преузима последњи снимљени податак (BuildingReading)
- Израчунава тренутну унутрашњу температуру користећи термодинамички модел
- Израчунава грешку регулације (разлика између жељене и тренутне температуре)
- Примењује П-регулатор: корекција = грешка × Kp (где је Kp = 1.5)
- Прилагођава снагу котлова додавањем корекције на тренутну снагу
- Ограничава снагу између 0 и максималне снаге котла
- Чува нову температуру и снагу у базу података
- Шаље SignalR обавештење frontend-u за real-time ажурирање

2. Снимање података (сваких 60 минута): Овај процес врши периодично снимање података за историјску анализу:

- Преузима тренутну спољну температуру од OpenWeatherMap API-ја
- За сваки објекат преузима последњи BuildingReading (критично за очување стања)
- Чува унутрашњу температуру из претходног записа (не ресетује је)
- Израчунава топлотне губитке на основу спољне температуре
- Израчунава потребну снагу за грејање
- Процењује дневне трошкове гријања
- Креира нови BuildingReading запис са свим подацима
- Чува запис у базу података

Термодинамички модел

Систем користи реалистичан термодинамички модел заснован на закону одржања енергије:

Q_net = Q_generated - Q_lost

гдје:

Q_generated = Σ(снага котла × ефикасност × 1000) [W]
Q_lost = топлотни губици објекта [W]

Промјена температуре се израчунава према формули:

ΔT = (Q_net × Δt) / (ThermalMass × Volume) × DampingFactor

гдје:
Δt = 60 секунди (временски корак)
ThermalMass = 3500 J/m^3·K (термичка маса објекта) (промјенљиво)
DampingFactor = 0.15 (фактор пригушења)

Нова унутрашња температура:

T_indoor_nova = T_indoor_stara + ΔT + OutdoorInfluence

7.3. Посебне функционалности по улози корисника

Функционалност / Радња	Администратор	Корисник
Регистрација и пријава	✓	✓
Креирање објеката	✕	✓
Креирање гасних котлова	✕	✓
Управљање сопственим котловима	✕	✓
Преглед свих корисника	✓	✕
Блокирање корисника	✓	✕
Подешавање глобалних параметара	✓	✕
Преглед дијаграма и потрошње	✓	✓
Преглед трошкова грејања	✓	✓
Извоз података (CSV)	✓	✓
Преглед основних информација	✓	✓

Корисник: Креирање и управљање објектима и котловима, преглед историјских дијаграма, анализа потрошње и трошкова, извоз података у CSV, real-time праћење температуре и снаге котлова, систем аларма за недовољан капацитет. **Администратор**: Управљање корисницима (блокирање, одблокирање), подешавање системских параметара (термичка маса, фактор сигурности, Kp регулатора, U-вредности), преглед свих објеката и котлова у систему, надзор над радом П-регулатора, дијагностика проблема.

8. Документација кода

8.1. Опис кључних модула

Controllers (Контролери)

- AuthController - Аутентикација и ауторизација (регистрација, пријава, JWT токени)
- UserController – CRUD са корисницима
- BuildingObjectsController - CRUD операције за објекте
- GasBoilersController - CRUD операције за котлове
- BuildingReadingsController - Преузимање историјских података
- SystemParametersController - Управљање глобалним параметрима (само Admin)
- AlarmsController - Управљање алармима

Services (Сервиси)

- BoilerControlBackgroundService - П-регулатор, извршава се сваки 1 минут
- DataRecordingBackgroundService - Снимање података, извршава се сваких 60 минута
- HistoricalDataService - Креирање BuildingReading ентитета, очување температуре
- BuildingCalculatorService - Прорачун топлотних губитака, потребне снаге, трошкова
- AlarmService - Провера капацитета, креирање аларма
- WeatherService - Интеграција са OpenWeatherMap API-јем

Repositories (Репозиторијуми)

- BuildingObjectRepository - Приступ BuildingObjects табели
- GasBoilerRepository - Приступ GasBoilers табели
- BuildingReadingRepository - Приступ BuildingReadings табели
- UserRepository - Приступ Users табели
- SystemParametersRepository - Приступ SystemParameters табели
- AlarmRepository - Приступ Alarms табели

8.2. Опис најважнијих класа и функција

BuildingObject (Класа ентитета) - Представља објекат који се грије. Садржи све неопходне податке за прорачун топлотних губитака и регулацију температуре.

BoilerControlBackgroundService (Главна класа П-регулатора) - Background service који се извршава сваки 1 минут. Представља главни циклус П-регулатора.

RegulateBuildingAsync - Примјењује П-регулатор на један објекат. Израчунава нову температуру, прилагођава снагу котлова, шаље SignalR обавештење.

CalculateRealisticIndoorTemperature - Термодинамички модел који израчунава нову унутрашњу температуру на основу топлотног биланса.

BuildingCalculatorService (Сервис за прорачуне)

- **CalculateHeatLoss** - Израчунава укупне топлотне губитке објекта кроз зидове, прозоре, плафон и под.

HistoricalDataService (Сервис за снимање података)

- **RecordBuildingDataAsync** - Креира нови BuildingReading запис.

8.3. Објашњење сложених алгоритама

П - Регулатор са термодинамиком

Проблем који ријешава: одржавање температуре и снаге котлова

- Чита тренутно стање (тренутне температуре, снаге, снимљени подаци)
- Термодинамички прорачун
- Рачуна се грешка
- Рачуна се укупно снага котлова
- Доноси се одлука о новој снази

Рачунање унутрашње температуре

- Узима се претходна унутрашњу температуру
- Израчуна генерисану топлоту
- Узима изгубљену топлоту и рачуна нето топлоту
- Промјена енергије и температуре
- Нова унутрашња температура

9. Потенцијална унапређења

9.1. Техничка унапређења

#	Унапређење	Опис и предности
1	**PID регулатор умјесто P**	Додавање интегралног (I) и диференцијалног (D) члана елиминисало би статичку грешку и осцилације. Систем би се брже стабилизовао и прецизније пратио задану температуру.
2	**Адаптивни Kp коефицијент**	Динамичка промјена коефицијента пропорционалности (Kp) у зависности од величине грешке, времена дана или спољне температуре — бољи одговор у екстремним условима.
3	**Реалнији физички модел**	Укључивање топлотне инерције зидова (time-lag), вишеслојне изолације и утицаја сунчевог зрачења дало би прецизнији симулациони резултат.
4	**Стварни API сензори**	Замјена симулираних вриједности индоор температуре стварним IoT сензорима преко MQTT или HTTP протокола.

9.2. Нове функционалности

#	Функционалност	Опис и предности
1	**Распоред гријања (Scheduling)**	Корисник дефинише временске зоне (нпр. ноћни режим 18°C, дневни 22°C). Систем аутоматски мјења desiredTemperature без ручне интервенције.
2	**Предвиђање температуре (ML)**	Коришћење историјских података и временске прогнозе за предвиђање будућих потреба за гријањем — бољи одговор прије него дође хладноћа.
3	**Мобилна апликација и push обавјештења**	React Native апликација са SignalR интеграцијом за праћење у реалном времену и аларме директно на телефон.
4	**Извјештаји и анализе**	Мјесечни PDF извјештаји о потрошњи гаса по згради, трошковима, ефикасности и историји аларма за менаџment.

9.3. Оптимизације

#	Оптимизација	Опис и предности
1	**Кеширање системских параметара**	Тренутно се sysParams учитава из базе при свакој итерацији. Имплементацијом IMemoryCache смањило би се оптерећење базе за ~40%.
2	**Паралелна регулација зграда**	Коришћење Parallel.ForEachAsync умјесто секвенцијалне петље — свака зграда регулисала би се паралелно, скраћујући укупно вријеме обраде.
3	**Batch upis у базу**	Умјесто засебног UpdateAsync за сваки бојлер, групни upis свих промјена у једном DbContext.SaveChangesAsync() позиву.
4	**Оптимизација SignalR порука**	Слање диференцијалних порука (само промјене) умјесто цјелокупног стања — мање мрежног саобраћаја, бржи frontend.

9.4. Могућности за скалабилност

Технологија	Сврха у скалабилном систему
Docker + Kubernetes	**Контејнеризација сервиса — лако деплојовање, аутоскалирање по оптерећењу, изолација компоненти.**
Redis Cache	**Кеширање температура и параметара у меморији — елиминација поновљених упита у базу, одговор < 1ms.**
RabbitMQ / Azure Service Bus	**Асинхрона комуникација између сервиса — Controller Service шаље поруке Alarm Service-у без чврсте спреге.**
TimescaleDB / InfluxDB	**Временска база података оптимизована за мјерења — брже упите за историјске температуре и аналитику.**
Azure IoT Hub	**Интеграција са стварним IoT сензорима и бојлерима преко MQTT протокола у индустријском окружењу.**

10. Закључак

У оквиру овог пројекта успјешно је реализован систем за управљање гасним бојлерима са потпуном backend и frontend инфраструктуром. Систем демонстрира примјену реалних инжењерских концепата у упрошћеном, али функционалном облику. Овај пројекат је добар шаблон за надоградњу у реалан систем, за реалну контролу гасних котлова.

Шта је постигнуто

- Имплементиран пропорционални (P) регулатор са deadband зоном и feedforward компензацијом — систем стабилно прати задану температуру.
- Физички модел загрјевања просторије симулира реалну термодинамику кроз параметре термалне масе, волумена и губитака топлоте.
- WebSocket комуникација (SignalR) омогућава праћење стања у реалном времену без поновног учитавања странице.
- Систем аларма аутоматски детектује прекорачење капацитета и критичне температурне одступе.
- Background Service аутономно регулише све зграде сваких 60 секунди без потребе за ручном интервенцијом.
- REST API са JWT ауторизацијом обезбјеђује сигуран приступ свим ресурсима система.